

UNIVERSIDAD DE GUADALAJARA

Centro Universitario de Ciencias Exactas e Ingenierías
División de Electrónica y Computación

REPORTE DE PRÁCTICA

Proyecto Final — Fase 2

Diseño de Datapath MIPS 32 bits — Instrucciones Tipo I y Buffers de Pipeline

Materia:	Arquitectura de Computadoras
Profesor:	Jorge Ernesto López Arce Delgado
Integrantes:	Diego Israel González Sánchez Derek Gabriel Casillas López Iván Samuel Ortiz Orellana Omar Luna Reyes
Fecha:	12 de mayo de 2026

1. INTRODUCCIÓN

1.1 Contexto y motivación — Fase 2

La Fase 1 del proyecto estableció un datapath MIPS de ciclo único capaz de ejecutar instrucciones Tipo R, validando el funcionamiento de todos los módulos hardware mediante simulación en ModelSim. En esta Fase 2 se amplía el repertorio de instrucciones del procesador para incluir el formato Tipo I, cubriendo operaciones con inmediato (`addi`, `slti`, `andi`, `ori`), acceso a memoria (`lw`, `sw`) y control de flujo condicional (`beq`).

Adicionalmente, el diseño incorpora los cuatro buffers de pipeline (IF/ID, ID/EX, EX/MEM, MEM/WB) requeridos por el capítulo 4.6 de Patterson y Hennessy, dejando la arquitectura preparada para la posterior implementación de segmentación completa. El sistema se valida con el programa de prueba especificado en el documento de Fase 2.

1.2 Formato Tipo I — Estructura de instrucción

Las instrucciones Tipo I utilizan 32 bits distribuidos en tres campos principales: el **opcode** de 6 bits identifica la operación, **rs** de 5 bits es el registro fuente, **rt** de 5 bits es el registro destino (o fuente secundario en `sw` y `beq`), e **imm** de 16 bits es el operando inmediato con extensión de signo.

Campo	opcode	rs	rt	imm / offset
Bits	31–26	25–21	20–16	15–0
Ancho	6 bits	5 bits	5 bits	16 bits
Función	Tipo de operación	Fuente 1	Destino / Fuente 2	Inmediato / Desplazamiento

1.3 Set de instrucciones implementado en Fase 2

La siguiente tabla presenta todas las instrucciones soportadas en el datapath de Fase 2, incluyendo las Tipo R ya implementadas y las nuevas Tipo I:

Instrucción	Tipo	Opcode	Funct / —	Sintaxis
add	R	000000	100000	add \$rd, \$rs, \$rt
sub	R	000000	100010	sub \$rd, \$rs, \$rt
and	R	000000	100100	and \$rd, \$rs, \$rt
or	R	000000	100101	or \$rd, \$rs, \$rt
xor	R	000000	100110	xor \$rd, \$rs, \$rt
nor	R	000000	100111	nor \$rd, \$rs, \$rt
slt	R	000000	101010	slt \$rd, \$rs, \$rt
sll	R	000000	000000	sll \$rd, \$rt, shamt
srl	R	000000	000010	srl \$rd, \$rt, shamt
nop	R	000000	000000	nop (sll \$0,\$0,0)
lw	I	100011	—	lw \$rt, offset(\$rs)

Instrucción	Tipo	Opcode	Funct / —	Sintaxis
sw	I	101011	—	sw \$rt, offset(\$rs)
beq	I	000100	—	beq \$rs, \$rt, label
addi	I	001000	—	addi \$rt, \$rs, imm
slti	I	001010	—	slti \$rt, \$rs, imm
andi	I	001100	—	andi \$rt, \$rs, imm
ori	I	001101	—	ori \$rt, \$rs, imm

2. OBJETIVO

2.1 Objetivo General

Extender el datapath MIPS de ciclo único desarrollado en la Fase 1 para soportar instrucciones Tipo I (`lw`, `sw`, `beq`, `addi`, `slli`, `andi`, `ori`), incorporar los cuatro buffers de pipeline requeridos para la segmentación futura, e implementar el programa de validación especificado, verificando los resultados esperados mediante simulación en ModelSim.

2.2 Objetivos Particulares

- Ampliar la Unidad de Control para decodificar los opcodes de las instrucciones Tipo I y generar correctamente la señal Branch para BEQ.
- Actualizar el módulo ALUControl para recibir un ALUOp de 3 bits que distingue entre las distintas operaciones inmediatas.
- Diseñar e implementar los cuatro buffers de pipeline (IF/ID, ID/EX, EX/MEM, MEM/WB) como registros síncronos con reset activo-alto, siguiendo el diseño del capítulo 4.6 de Patterson y Hennessy.
- Agregar el sumador de dirección de salto BEQ y la lógica de selección del siguiente PC (`pc_src = Branch AND zero`).
- Codificar manualmente el programa de validación de Fase 2 en binario (`TestF2_MemInst.mem`) y verificar los resultados esperados: `$t0=10`, `$t1=1`, `$t2=0`, `$t3=10`, `$t4=10`, `mem[0]=10`.
- Codificar el algoritmo de primalidad propuesto en Fase 1 en código máquina binario, aprovechando las nuevas instrucciones Tipo I disponibles.

3. DESARROLLO

3.1 Arquitectura del datapath — Fase 2

El datapath de Fase 2 mantiene la naturaleza de ciclo único de Fase 1 e incorpora dos extensiones principales: soporte para instrucciones Tipo I y los buffers de pipeline intercalados entre cada par de etapas del flujo **IF** → **ID** → **EX** → **MEM** → **WB**. Aunque en esta fase el procesador opera como single-cycle (todos los buffers se actualizan en cada ciclo), la presencia de los registros IF/ID, ID/EX, EX/MEM y MEM/WB establece la infraestructura necesaria para activar la segmentación en la Fase siguiente.

Los módulos que no sufrieron modificaciones respecto a la Fase 1 son: PC, Adder, ALU, Mux2, SignExtend, RegisterFile, DataMemory y TestF1_BReg.mem. Los módulos modificados o nuevos se describen a continuación.

3.2 Módulo Control — Unidad de Control Principal

La señal **ALUOp** se amplió de 2 a 3 bits para distinguir las siete operaciones distintas que se pueden requerir de la ALU. Se añadió la señal de salida **Branch** que se activa únicamente para la instrucción beq. La tabla de verdad del módulo ahora cubre ocho opcodes:

Instrucción	opcode	RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp
Tipo R	00000 0	1	0	0	1	0	0	0	010
lw	10001 1	0	1	1	1	1	0	0	000
sw	10101 1	0	1	0	0	0	1	0	000
beq	00010 0	0	0	0	0	0	0	1	001
addi	00100 0	0	1	0	1	0	0	0	000
slti	00101 0	0	1	0	1	0	0	0	011
andi	00110 0	0	1	0	1	0	0	0	100
ori	00110 1	0	1	0	1	0	0	0	101

```
// Fragment: Control.v — case(opcode)
6'b000000: begin // Tipo R
    RegDst=1; RegWrite=1; ALUOp=3'b010; end
6'b100011: begin // LW
    ALUSrc=1; MemtoReg=1; RegWrite=1; MemRead=1; ALUOp=3'b000; end
6'b101011: begin // SW
    ALUSrc=1; MemWrite=1; ALUOp=3'b000; end
6'b000100: begin // BEQ
    Branch=1; ALUOp=3'b001; end
6'b001000: begin // ADDI
    ALUSrc=1; RegWrite=1; ALUOp=3'b000; end
```

```

6'b001010: begin // SLTI
    ALUSrc=1; RegWrite=1; ALUOp=3'b011; end
6'b001100: begin // ANDI
    ALUSrc=1; RegWrite=1; ALUOp=3'b100; end
6'b001101: begin // ORI
    ALUSrc=1; RegWrite=1; ALUOp=3'b101; end

```

3.3 Módulo ALUControl — Control de la ALU

Se actualizó la entrada **ALUOp** de 2 a 3 bits. Los nuevos casos cubren directamente la operación ALU requerida por cada instrucción Tipo I, sin necesidad de consultar el campo `funct`. El caso 3'b010 sigue siendo el de instrucciones Tipo R, que continúa usando el campo `funct` para seleccionar la operación exacta.

ALUOp[2:0]	alu_control[3:0]	Operación	Instrucciones que lo usan
010	usa funct	Ver tabla Tipo R	add, sub, and, or, xor, nor, slt, sll, srl
000	0010	ADD	lw, sw, addi
001	0110	SUB	beq
011	0111	SLT	slti
100	0000	AND	andi
101	0001	OR	ori

3.4 Módulo PipelineBuffers — Cuatro buffers de pipeline

Se implementaron los cuatro buffers de pipeline en el archivo `PipelineBuffers.v`. Cada buffer es un registro síncrono de flanco positivo con reset activo-alto que lleva todas sus salidas a cero. Todos comparten el mismo **CLK** y la misma señal **reset** que el PC, conforme a la Figura 2 del documento de Fase 2.

- IF/ID Buffer: captura PC+4 e instrucción de 32 bits al finalizar la etapa IF.
- ID/EX Buffer: captura las ocho señales de control (RegDst, ALUSrc, MemtoReg, RegWrite, MemRead, MemWrite, Branch, ALUOp[2:0]) más los datos de registros, el inmediato extendido, y los campos rs, rt, rd, shamt y funct.
- EX/MEM Buffer: captura la señal zero de la ALU, el resultado de la ALU, el dato a escribir en memoria, la dirección de salto BEQ calculada y las señales de control de la etapa MEM.
- MEM/WB Buffer: captura el dato leído de la memoria de datos y el resultado de la ALU para seleccionar cuál se escribe de vuelta en el banco de registros.

```

// Fragment: IF_ID_Buffer
always @(posedge clk or posedge reset) begin
    if (reset) begin
        pc_plus4_out    <= 32'd0;
        instruction_out <= 32'd0;
    end else begin
        pc_plus4_out    <= pc_plus4_in;
        instruction_out <= instruction_in;
    end
end
end

```

3.5 Módulo MIPS_Top — Datapath completo Fase 2

El módulo superior MIPS_Top fue reescrito completamente para integrar los buffers de pipeline y la lógica de control de flujo BEQ. El flujo de ejecución de una instrucción Tipo I sigue las mismas etapas que Tipo R, con las siguientes diferencias clave:

- MUX ALUSrc (en EX): selecciona el inmediato extendido con signo en lugar del segundo registro cuando ALUSrc=1.
- MUX RegDst (en EX): selecciona rt como registro destino (campo [20:16]) en lugar de rd ([15:11]) cuando RegDst=0.
- Adder BEQ (en EX): calcula PC+4 + (imm_extendido << 2) para obtener la dirección de salto.
- Lógica BEQ (en MEM): pc_src = Branch_mem AND zero_mem. Si el resultado es 1, el MUX de PC selecciona la dirección de salto en lugar de PC+4.
- MUX MemtoReg (en WB): selecciona el dato leído de memoria cuando MemtoReg=1 (instrucción lw).

```
// Fragment: MIPS_Top - lógica BEQ y selección de PC

// Cálculo de dirección de salto en etapa EX:
Adder adder_branch(
    .a(pc_plus4_ex),
    .b({sign_ext_imm_ex[29:0], 2'b00}), // imm << 2
    .result(branch_target_ex)
);

// Señal de salto en etapa MEM:
assign pc_src = Branch_mem & zero_mem;

// MUX de selección de siguiente PC en etapa IF:
Mux2 #(32) mux_next_pc(
    .d0(pc_plus4_if), // flujo normal
    .d1(branch_target_mem), // salto BEQ
    .sel(pc_src),
    .out(next_pc)
);
```

3.6 Archivo TestF2_MemInst.mem — Programa de validación

El programa de validación corresponde exactamente a la Figura 1 del documento de Fase 2. Cada instrucción fue codificada manualmente en binario de 32 bits siguiendo el formato Tipo I estándar MIPS. La instrucción beq \$zero,\$zero,.inicio utiliza un offset de -8 (1111111111111000 en complemento a dos de 16 bits) para saltar de la instrucción [7] de regreso a la instrucción [0], creando el loop infinito especificado.

Índice	Ensamblador	Codificación binaria (32 bits)
[0]	nop	00000000000000000000000000000000
[1]	addi \$t0, \$zero, 10	00100000000010000000000000001010
[2]	sli \$t1, \$zero, 10	00101000000010010000000000001010
[3]	andi \$t2, \$zero, 10	00110000000010100000000000001010
[4]	ori \$t3, \$zero, 10	00110100000010110000000000001010
[5]	sw \$t0, 0(\$zero)	10101100000010000000000000000000

Índice	Ensamblador	Codificación binaria (32 bits)
[6]	lw \$t4, 0(\$zero)	1000110000000110000000000000000000
[7]	beq \$zero,\$zero,.inicio	000100000000000000111111111111000
[8]	nop (nunca alcanzado)	00000000000000000000000000000000

Resultados esperados de simulación:

Registro / Memoria	Valor decimal esperado	Instrucción que lo produce
\$t0 (reg 8)	10	addi \$t0, \$zero, 10
\$t1 (reg 9)	1	slti \$t1, \$zero, 10 (0 < 10 → 1)
\$t2 (reg 10)	0	andi \$t2, \$zero, 10 (0 AND 10 = 0)
\$t3 (reg 11)	10	ori \$t3, \$zero, 10 (0 OR 10 = 10)
mem[0]	10	sw \$t0, 0(\$zero)
\$t4 (reg 12)	10	lw \$t4, 0(\$zero)

4. PROGRAMA ENSAMBLADOR — TEST DE PRIMALIDAD

4.1 Descripción del algoritmo

El algoritmo propuesto en el Reporte de Fase 1 —Test de Primalidad por Restas Sucesivas— puede ahora implementarse completamente en hardware gracias a las instrucciones Tipo I disponibles en Fase 2. El algoritmo determina si un número *n* es primo probando divisores desde 2 hasta *n*-1: para cada divisor calcula el residuo de *n* ÷ divisor mediante restas sucesivas; si algún residuo es cero, el número no es primo; si ningún divisor produce residuo cero, el número es primo.

En Fase 1, este algoritmo no podía ejecutarse porque requería *addi* para inicializar registros con constantes y *beq* para implementar los bucles de control. Ambas instrucciones están disponibles a partir de Fase 2.

4.2 Asignación de registros

Registro MIPS	Número	Uso en el algoritmo
\$zero	0	Constante cero (siempre)
\$v0	2	Resultado: 1 = primo, 0 = no primo
\$t0	8	<i>n</i> — número a evaluar (valor = 7)
\$t1	9	divisor — empieza en 2, incrementa de 1 en 1
\$t2	10	residuo — copia de <i>n</i> , se resta el divisor iterativamente
\$t3	11	bandera temporal (resultado de <i>slti</i>)
\$t4	12	constante 1 (para comparaciones)

4.3 Código ensamblador

```
# Test de Primalidad por Restas Sucesivas
# Entrada: n = 7 (cargado por addi)
# Salida:  $v0 = 1 (primo) / 0 (no primo)

        nop                # [0]  nop inicial
        addi $t0, $zero, 7  # [1]  n = 7
        addi $t1, $zero, 2  # [2]  divisor = 2
        addi $t4, $zero, 1  # [3]  constante 1
        addi $v0, $zero, 1  # [4]  resultado = 1 (asumimos primo)

loop_div:
        slti $t3, $t1, 7    # [5]  t3 = (divisor < n)?
        beq  $t3, $zero, fin # [6]  si no → divisor >= n → es primo

init_mod:
        add  $t2, $t0, $zero # [7]  residuo = n

loop_mod:
        slti $t3, $t2, 1    # [8]  t3 = (residuo < 1)?
        beq  $t3, $zero, sub_mod # [9]  t3==0 → residuo >= 1 → seguir restando
        beq  $t2, $zero, no_primo # [10] residuo==0 → divisible → no primo
        beq  $zero, $zero, next_div # [11] residuo<1 y !=0 → siguiente divisor
```

```

sub_mod:
    sub    $t2, $t2, $t1          # [12] residuo -= divisor
    beq    $zero, $zero, loop_mod # [13] volver a loop_mod

next_div:
    addi   $t1, $t1, 1           # [14] divisor++
    beq    $zero, $zero, loop_div # [15] volver a loop_div

no_primo:
    addi   $v0, $zero, 0         # [16] resultado = 0 (no es primo)

fin:
    nop                                # [17] nop
    beq    $zero, $zero, fin      # [18] halt (loop infinito)

```

4.4 Codificación en código máquina

Cada instrucción fue codificada manualmente en binario de 32 bits. Los offsets de los BEQ se calcularon como (dirección_destino - (PC_instr+1)):

Índice	Ensamblador	Codificación binaria (32 bits)
[0]	nop	00000000000000000000000000000000
[1]	addi \$t0, \$zero, 7	00100000000010000000000000000111
[2]	addi \$t1, \$zero, 2	00100000000010010000000000000010
[3]	addi \$t4, \$zero, 1	00100000000011000000000000000001
[4]	addi \$v0, \$zero, 1	00100000000000100000000000000001
[5]	slti \$t3, \$t1, 7 (loop_div)	00101001001010110000000000000111
[6]	beq \$t3,\$zero,fin (+10)	00010001011000000000000000001010
[7]	add \$t2,\$t0,\$zero (init_mod)	00000001000000000101000000100000
[8]	slti \$t3,\$t2,1 (loop_mod)	00101001010010110000000000000001
[9]	beq \$t3,\$zero,sub_mod (+2)	00010001011000000000000000000010
[10]	beq \$t2,\$zero,no_primo (+5)	00010001010000000000000000000101
[11]	beq \$zero,\$zero,next_div (+2)	00010000000000000000000000000010
[12]	sub \$t2,\$t2,\$t1 (sub_mod)	00000001010010010101000000100010
[13]	beq \$zero,\$zero,loop_mod (-6)	00010000000000000111111111111010
[14]	addi \$t1,\$t1,1 (next_div)	00100001001010010000000000000001
[15]	beq \$zero,\$zero,loop_div (-11)	000100000000000001111111111110101
[16]	addi \$v0,\$zero,0 (no_primo)	00100000000000010000000000000000
[17]	nop (fin)	00000000000000000000000000000000
[18]	beq \$zero,\$zero,fin (-2) halt	00010000000000000111111111111110

4.5 Traza de ejecución — ejemplo n = 7

Para n = 7 (número primo), el algoritmo verifica divisores 2, 3, 4, 5 y 6. Ninguno produce residuo cero:

Ciclo externo	Divisor (\$t1)	Residuo final (\$t2)	¿Divisible?	Resultado
1	2	1 (7-2-2-2 = 1)	No	Continúa
2	3	1 (7-3-3 = 1)	No	Continúa
3	4	3 (7-4 = 3)	No	Continúa
4	5	2 (7-5 = 2)	No	Continúa
5	6	1 (7-6 = 1)	No	Continúa
6	7	divisor ≥ n	—	FIN: \$v0 = 1 (PRIMO)

5. CONCLUSIONES

Diego Israel González Sánchez

La extensión del datapath para soportar instrucciones Tipo I evidenció con claridad la diferencia entre el diseño de la unidad de control de Fase 1, que solo necesitaba activar señales para un único opcode, y el de Fase 2, donde cada opcode genera una combinación distinta de señales. La decisión de ampliar ALUOp a 3 bits fue clave para mantener el diseño modular: el módulo ALUControl sigue siendo el único punto donde se decide la operación de la ALU, sin necesidad de propagar el opcode original por todo el datapath. Esta separación de responsabilidades es un principio de diseño que se traslada directamente a la práctica de ingeniería de hardware.

Derek Gabriel Casillas López

Implementar los buffers de pipeline fue la tarea que más claridad aportó sobre la diferencia entre un procesador de ciclo único y uno segmentado. En apariencia son simples registros, pero al colocarlos entre etapas se comprende que cada buffer debe capturar exactamente las señales que la etapa siguiente necesita, y nada más. Codificar manualmente el programa de validación en binario —calcular los offsets de los BEQ, verificar los campos `rt` vs `rd` según el tipo de instrucción— consolidó la comprensión del formato Tipo I a un nivel que ninguna lectura teórica puede igualar.

Iván Samuel Ortiz Orellana

La implementación de la lógica BEQ resultó ser el componente más instructivo de esta fase. La señal `pc_src` es el resultado de un AND entre `Branch` y `zero`: dos señales generadas en etapas distintas del pipeline (ID y EX respectivamente) que deben estar disponibles simultáneamente en la etapa MEM para tomar la decisión de salto. Esto anticipa precisamente el tipo de problema de dependencias de datos y control que se resolverá con técnicas de forwarding y detección de hazards en el pipeline completo. La experiencia de diseñar el datapath bufferizado deja claro por qué el pipeline requiere una unidad de detección de riesgos.

Omar Luna Reyes

Codificar el algoritmo de primalidad en ensamblador MIPS y traducirlo manualmente a código máquina binario completó el ciclo iniciado en Fase 1, donde el algoritmo solo podía proponerse teóricamente. Verificar la traza de ejecución para $n = 7$ —siguiendo ciclo por ciclo cómo los registros `$t1`, `$t2` y `$t3` cambian de valor— refuerza la comprensión de cómo los bucles y las condiciones de alto nivel se traducen en instrucciones de comparación, salto condicional y salto incondicional en el ISA MIPS. La posibilidad de verificar el resultado automáticamente en el testbench demuestra la utilidad de la simulación como herramienta de validación.

Conclusión General

La Fase 2 del proyecto completó la transición de un procesador capaz únicamente de operaciones entre registros a un datapath funcional que interactúa con memoria y soporta control de flujo condicional. La incorporación de los buffers de pipeline prepara la arquitectura para la segmentación, que es el paso natural hacia un procesador de alto rendimiento. El proyecto demuestra que la arquitectura MIPS, diseñada con principios RISC de ortogonalidad y simplicidad, permite agregar funcionalidad de forma incremental y sistemática, validando cada extensión por separado antes de integrarla al sistema completo.

6. REFERENCIAS

- [1] D. A. Patterson y J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 5.^a ed. Waltham, MA: Morgan Kaufmann, 2014.
- [2] MIPS Technologies, *MIPS32 Architecture for Programmers Volume II: The MIPS32 Instruction Set*. Mountain View, CA: MIPS Technologies, 2001.
- [3] S. Brown y Z. Vranesic, *Fundamentals of Digital Logic with Verilog Design*, 3.^a ed. New York, NY: McGraw-Hill, 2014.
- [4] Intel/Altera, *ModelSim-Altera Software Simulation User Guide*. San Jose, CA: Altera Corporation, 2016.
- [5] D. M. Harris y S. L. Harris, *Digital Design and Computer Architecture*, 2.^a ed. Waltham, MA: Morgan Kaufmann, 2012.